

Chapter 14: Feature Reinforcement Learning

An Introduction to Universal Artificial Intelligence

Mapping Histories to States: Φ MDP, Extreme State Aggregation, and CTMRL

Marcus Hutter, Elliot Catt, David Quarel

Taylor & Francis / CRC Press, 2024

Chapter Overview

- 1 Introduction: An Alternative to AIXI
- 2 14.1 Feature Reinforcement Learning Setup
- 3 14.2 History Aggregation beyond MDPs
 - 14.2.1 Surrogate MDP and Dispersion Probability
 - 14.2.2 (Q-)Value Inheritance for Fixed and Optimal Policy
 - 14.2.3 Extreme State Aggregation
- 4 14.3 Feature MDP
 - 14.3.1 Feature Learning
 - 14.3.2 Choices of the Cost Function
 - 14.3.3 Feature Dynamic Bayesian Networks
- 5 14.4 Context Tree Maximization Reinforcement Learning
- 6 14.5 Exercises
- 7 14.6 History and References

Outline

- 1 Introduction: An Alternative to AIXI
- 2 14.1 Feature Reinforcement Learning Setup
- 3 14.2 History Aggregation beyond MDPs
 - 14.2.1 Surrogate MDP and Dispersion Probability
 - 14.2.2 (Q-)Value Inheritance for Fixed and Optimal Policy
 - 14.2.3 Extreme State Aggregation
- 4 14.3 Feature MDP
 - 14.3.1 Feature Learning
 - 14.3.2 Choices of the Cost Function
 - 14.3.3 Feature Dynamic Bayesian Networks
- 5 14.4 Context Tree Maximization Reinforcement Learning
- 6 14.5 Exercises
- 7 14.6 History and References

From General Reinforcement Learning to MDPs

Throughout this book we have mostly used a **Bayesian approach** to the general reinforcement learning problem, culminating in the (often incomputable) agent AIXI and its approximations. This chapter presents an **alternative** approach called **Feature Reinforcement Learning (FRL)**, which instead tries to *map* the hard general reinforcement learning problem down to an easier problem that we already know how to solve well: a **Markov decision process (MDP)**.

FRL can be viewed as a more practical substitute for the chronological Solomonoff distribution (Theorem 7.4.5), one that avoids the expensive expectimax planning used in the Bayesian approach.

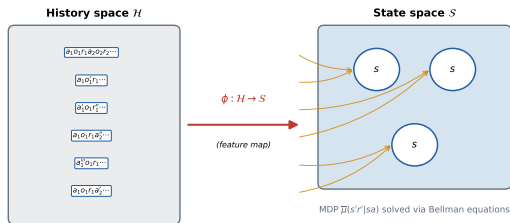
The core idea. We construct a **feature map** that sends each *history* (everything the agent has seen and done so far) to a *state* in some smaller state space. This mapping turns both the agent and the environment into objects that depend on states instead of full histories. If histories that are “similar enough” get mapped to the same state, solving the resulting MDP can still yield a near-optimal agent for the original, much harder problem.

Questions this chapter answers. When are two histories similar? What makes a feature map “good”? How do we find one efficiently? And how does performing well on the small state space translate back into performing well on the original history space?

Two Criteria for a Good Feature Map

This chapter lays out the general FRL framework (Section 14.1) and then studies **two fundamentally different criteria** for judging a feature map ϕ :

- **Section 14.2 – History aggregation beyond MDPs.** We do *not* require that the mapped process itself be Markov (or even approximately so). Instead we only ask that histories with *close* (Q -)Values get mapped to the same state. This is called **extreme state aggregation**, and remarkably it still lets us transfer a good policy back to the original problem.
- **Sections 14.3–14.4 – Feature MDP and CTMRL.** We *do* require the mapped process to be (approximately) an MDP, and search for the smallest such state space using a **cost function** inspired by the minimum description length (MDL) principle. Section 14.4 instantiates this idea concretely using **context trees**, giving the practical algorithm **CTMRL**.



Outline

- 1 Introduction: An Alternative to AIXI
- 2 14.1 Feature Reinforcement Learning Setup
- 3 14.2 History Aggregation beyond MDPs
 - 14.2.1 Surrogate MDP and Dispersion Probability
 - 14.2.2 (Q-)Value Inheritance for Fixed and Optimal Policy
 - 14.2.3 Extreme State Aggregation
- 4 14.3 Feature MDP
 - 14.3.1 Feature Learning
 - 14.3.2 Choices of the Cost Function
 - 14.3.3 Feature Dynamic Bayesian Networks
- 5 14.4 Context Tree Maximization Reinforcement Learning
- 6 14.5 Exercises
- 7 14.6 History and References

Feature Maps: From Histories to States

The Feature Map ϕ

A **feature map** is a function

$$\phi : \mathcal{H} \rightarrow \mathcal{S}, \quad \mathcal{H} := (\mathcal{E} \times \mathcal{A})^* \times \mathcal{E}$$

which maps a history space $\mathcal{H}$ to a state space $\mathcal{S}$, turning our problem from a general reinforcement problem into a **Markov decision process (MDP)** over the state space $\mathcal{S}$.

Plain English. $\mathcal{E} = \mathcal{O} \times \mathcal{R}$ (*observation space* $\times$ *reward space*) is the **percept space** – what the agent perceives at each step – and $\mathcal{A}$ is the **action space**. So $\mathcal{H}$ is the set of all finite sequences of actions and percepts. The map ϕ throws away everything about a history except which **state** $s \in \mathcal{S}$ it belongs to. Unless stated otherwise, $\mathcal{S}$ is assumed finite, and ideally small (in some sense we make precise later).

Notation convention. In this chapter, π (pi) and μ (mu) denote the **policy** and **environment** in the original general reinforcement learning setting, while $\bar{\pi}$ and $\bar{\mu}$ (“pi-bar”, “mu-bar”) denote the policy and environment of the **derived MDP** setting.

History-Based (Q-)Values

The goal of FRL is to use solutions found in the simplified MDP/state space setting to build agents with good behavior in the *original* general reinforcement learning setting. We already know what “good behavior” means there: maximizing the discounted expected future reward sum, i.e. the **value** (Definition 6.6.1) and **Q-value** (Definition 6.7.1). In this chapter we use *unnormalized* infinite-horizon (Q-)Values with geometric discounting in the true environment μ :

(Q-)Value Functions in the General Setting

$$V_{\mu}^{\pi}(h_{<t}) := \mathbb{E}_{\mu}^{\pi} \left[\sum_{k=t}^{\infty} \gamma^{k-t} r_k \mid h_{<t} \right] = \sum_{a_t} \pi(a_t | h_{<t}) Q_{\nu}^{\pi}(h_{<t}, a_t)$$

$$Q_{\mu}^{\pi}(h_{<t}, a_t) := \mathbb{E}_{\mu}^{\pi} \left[\sum_{k=t}^{\infty} \gamma^{k-t} r_k \mid h_{<t}, a_t \right] = \sum_{e_t} \nu(e_t | h_{<t}, a_t) [r_t + \gamma V_{\mu}^{\pi}(h_{1:t})]$$

Plain English. γ (gamma) is the **discount factor** (a number in $(0, 1)$ making distant rewards count less); r_k is the reward at time k ; $\mathbb{E}_{\mu}^{\pi}[\cdot | h_{<t}]$ is the expectation under policy π in μ , given history $h_{<t}$. $V_{\mu}^{\pi}(h_{<t})$ is the value of following π from now on; $Q_{\mu}^{\pi}(h_{<t}, a_t)$ is the value of taking a_t now, then following π . The optimal policy is $\pi_{\mu}^* \in \arg \max_{\pi} V_{\mu}^{\pi}(\epsilon)$.

Markov Decision Processes, Recalled

Markov Environment

An environment $\bar{\mu}$ is a **Markov Decision Process (MDP)** if it depends only on the most recent observation and action. If the environment is Markov, we call the observations **states** (notation s , set $\mathcal{S}$), and require for all histories $h_{<t}$, states s_t , and rewards r_t :

$$\bar{\mu}(s_t r_t | h_{<t}) = \bar{\mu}(s_t r_t | s_{t-1} a_{t-1})$$

Plain English. This says the probability of the next state/reward pair depends *only* on the immediately preceding state and action – not on how we got there. If the state is fully observed ($o_t = s_t$, as in Chapter 11), this is the classical **completely observable MDP** setting. Here, though, the state space $\mathcal{S}$ for $\bar{\mu}$ is kept separate from the observation space $\mathcal{O}$ for μ : we abuse notation slightly and write $\bar{\mu} : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S} \times \mathcal{R})$ (Δ denotes “probability distribution over”). We use $\bar{\pi}$ for a **deterministic** Markov policy, which depends only on the previous state.

Bellman (Optimality) Equations

Because the MDP $\bar{\mu}$ and policy $\bar{\pi}$ depend only on the state (and are time-invariant), it is convenient to introduce the shorthand $s = s_{t-1}$, $a = a_t$, $s' = s_t$, $r' = r_t$.

Bellman Equations (Theorem 6.7.2, specialized to MDPs) – Eq. (14.1.1)

$$Q_{\bar{\mu}}^{\bar{\pi}}(s, a) = \sum_{s', r' \in \mathcal{S} \times \mathcal{R}} \bar{\mu}(s' r' | sa) [r' + \gamma V_{\bar{\mu}}^{\bar{\pi}}(s')], \quad V_{\bar{\mu}}^{\bar{\pi}}(s) = Q_{\bar{\mu}}^{\bar{\pi}}(s, \bar{\pi}(s))$$

Bellman Optimality Equations – Eq. (14.1.2)

$$Q_{\bar{\mu}}^*(s, a) = \sum_{s', r'} \bar{\mu}(s' r' | sa) [r' + \gamma V_{\bar{\mu}}^*(s')], \quad V_{\bar{\mu}}^*(s) = \max_a Q_{\bar{\mu}}^*(s, a), \quad \bar{\pi}^*(s) \in \arg \max_a Q_{\bar{\mu}}^*(s, a)$$

Plain English. The value of a state equals the reward you get plus the (discounted) value of wherever you land next, averaged over all possible next state/reward pairs (s', r') weighted by their probability $\bar{\mu}(s' r' | sa)$. The **optimal** policy $\bar{\pi}^*$ simply always picks the action a with the highest $Q_{\bar{\mu}}^*$ at the current state. These equations only hold for *deterministic* $\bar{\pi}$, but since we care about optimal policies – which can always be chosen deterministic (Definition 6.6.1) – this is not a real restriction.

Python: Bellman Equations on a Toy 2-State MDP

We solve the Bellman optimality equations (14.1.2) by value iteration on a hand-built 2-state MDP $\mathcal{S} = \{0, 1\}$, $\mathcal{A} = \{\text{stay}, \text{go}\}$, with $\gamma = 0.9$.

```
1 gamma = 0.9
2 S, A = [0, 1], ["stay", "go"]
3 P = {0: {"stay": {0: .8, 1: .2}, "go": {0: .3, 1: .7}},
4       1: {"stay": {0: .1, 1: .9}, "go": {0: .6, 1: .4}}}
5 R = {0: {"stay": 1.0, "go": 0.0}, 1: {"stay": 2.0, "go": 5.0}}
6
7 V = {0: 0.0, 1: 0.0}
8 for it in range(2000):
9     newV = {s: max(R[s][a] + gamma * sum(P[s][a][sp] * V[sp] for sp in S)
10                for a in A) for s in S}
11     if max(abs(newV[s] - V[s]) for s in S) < 1e-10:
12         V = newV; break
13     V = newV
14 Q = {s: {a: R[s][a] + gamma * sum(P[s][a][sp] * V[sp] for sp in S)
15        for a in A} for s in S}
16 pi = {s: max(A, key=lambda a: Q[s][a]) for s in S}
17
18 # Output: converged after 229 iterations
19 # V*(0)=24.8031 V*(1)=28.7402; Q*(0,go)=24.8031 Q*(1,go)=28.7402
20 # pi*(0) = go pi*(1) = go
```

V^* is the fixed point of taking the best action's Q^* in every state.

Outline

- 1 Introduction: An Alternative to AIXI
- 2 14.1 Feature Reinforcement Learning Setup
- 3 14.2 History Aggregation beyond MDPs
 - 14.2.1 Surrogate MDP and Dispersion Probability
 - 14.2.2 (Q-)Value Inheritance for Fixed and Optimal Policy
 - 14.2.3 Extreme State Aggregation
- 4 14.3 Feature MDP
 - 14.3.1 Feature Learning
 - 14.3.2 Choices of the Cost Function
 - 14.3.3 Feature Dynamic Bayesian Networks
- 5 14.4 Context Tree Maximization Reinforcement Learning
- 6 14.5 Exercises
- 7 14.6 History and References

Extreme State Aggregation: The Idea

Extreme state aggregation [Hut16] is the idea of aggregating histories which have *close* (Q-)Values. “Aggregation” means a many-to-one mapping from histories to states. If we can map (aggregate) histories with close (Q-)Values to the same state, then solving this new *state* problem should give a good solution to the original problem.

This is in contrast to many other aggregation methods, which try to map similar histories to states based on the *contents* of the histories (e.g., histories with the same most recent observation get the same state). Most remarkably, we do *not* even need to require that the history process μ maps (even approximately) to an MDP – we will instead build a **surrogate** $\bar{\mu}$ that behaves as if it were one.

This section shows how such aggregation can lead to well-performing agents, and concludes with an explicit construction of a ϕ that aggregates this way (**extreme state aggregation**, Section 14.2.3).

Definition 14.2.1: The Feature Environment

Given an observation space $\mathcal{O}$, action space $\mathcal{A}$, reward space $\mathcal{R}$, environment μ , state space $\mathcal{S}$, and feature map $\phi : \mathcal{H} \rightarrow \mathcal{S}$, we construct a **feature environment** μ_ϕ . Let $h \in \mathcal{H}$ be a history of *any* length (unlike $h_{<t}$, which has length t); if h has length t , then $h' = hao'r'$ is its extension to length $t+1$.

Definition 14.2.1 (Feature Environment μ_ϕ)

$$\mu_\phi(s'r'|ha) := \sum_{o': \phi(hao'r')=s'} \mu(o'r'|ha)$$

Plain English. μ_ϕ answers: “given the true environment μ , what is the probability of landing in state s' (and getting reward r'), after taking action a ?” We compute it by summing the true probability $\mu(o'r'|ha)$ over every observation o' that, once appended to the history and mapped through ϕ , would land us in s' .

We have a simple condition for when μ_ϕ happens to already be an MDP: this holds if there is an MDP kernel $p : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S} \times \mathcal{R})$ such that $\mu_\phi(s'r'|ha) = p(s'r'|sa)$ whenever $\phi(h) = s$. The key difference to most other state-aggregation methods is that we will *not* require μ_ϕ to be an MDP (not even approximately).

Definition 14.2.2: Dispersion Probability

Instead we construct a surrogate MDP using a kind of stochastic *inverse* of ϕ .

Definition 14.2.2 (Dispersion Probability B)

Let $B : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{H})$ be a probability distribution on histories for each state-action pair, such that $B(h|sa) = 0$ if $s \neq \phi(h)$. The formal constraints are

$$B(h|sa) \geq 0 \quad \text{and} \quad \sum_{h \in \mathcal{H}} B(h|sa) = \sum_{h: \phi(h)=s} B(h|sa) = 1 \quad \forall s, a$$

Plain English. B may be viewed as a stochastic “inverse” to ϕ : it assigns non-zero probability *only* to histories h that ϕ maps to s (i.e. $h \in \phi^{-1}(s)$), and for each state-action pair those probabilities sum to 1. Intuitively, $B(h|sa)$ says “if I am told the current state is s and I took action a , how likely is it that the actual underlying history was h ?”

Crucially, the sum defining B ranges over histories of *all* lengths, making B a discrete probability over the countable space $\mathcal{H}$ – unlike the “continuous” probability measure $\mu(h) := \mu(\Gamma_h)$ over infinite histories defined via cylinder sets Γ_h .

Definition 14.2.3: The Surrogate MDP

Definition 14.2.3 (Surrogate MDP)

Given feature environment μ_ϕ and dispersion probability B , we define the **surrogate MDP**:

$$\bar{\mu}(s' r' | sa) := \sum_{h \in \mathcal{H}} \mu_\phi(s' r' | ha) B(h | sa)$$

Plain English. To predict the next state/reward from (s, a) under the surrogate MDP, we average the feature environment's prediction $\mu_\phi(s' r' | ha)$ over all histories h that could be “hiding behind” state s , weighted by how likely B thinks each one is.

Remark 14.2.4 (If μ_ϕ is an MDP, then $\bar{\mu} = \mu_\phi$)

The surrogate-MDP notion is consistent: if μ_ϕ is already an MDP, then $\bar{\mu} = \mu_\phi$ for *any* choice of dispersion probability B , since

$$\bar{\mu}(s' r' | sa) \equiv \sum_{h: \phi(h)=s} \mu_\phi(s' r' | ha) B(h | sa) = \mu_\phi(s' r' | sa) \sum_{h: \phi(h)=s} B(h | sa) = \mu_\phi(s' r' | sa)$$

where the second equality holds iff μ_ϕ is an MDP – an assumption we will *not* make in general.

Lemma 14.2.5: Dispersion Probability Equivalence I

Lemma 14.2.5 (Dispersion Probability Equivalence)

For any function $f : \mathcal{S} \times \mathcal{R} \rightarrow \mathbb{R}$, any feature map ϕ , feature function μ_ϕ (Definition 14.2.1), surrogate MDP $\bar{\mu}$ (Definition 14.2.3), with $s' := \phi(h')$ and $h' = hao'r'$, it holds

$$\sum_{h \in \mathcal{H}} B(h|sa) \sum_{o', r'} \mu(o' r' | ha) f(s', r') = \sum_{s', r'} \bar{\mu}(s' r' | sa) f(s', r')$$

Plain English. This says that averaging any function f of the *true* next state/reward over B and μ gives exactly the same answer as averaging f directly over the surrogate MDP $\bar{\mu}$. In other words, $\bar{\mu}$ is a faithful summary of μ , for the purposes of computing expectations, once we condition on s, a via B .

Lemma 14.2.5: Dispersion Probability Equivalence II

Proof.

$$\begin{aligned} & \sum_{h \in \mathcal{H}} B(h|sa) \sum_{o', r'} \mu(o' r' | ha) f(s', r') \\ & \stackrel{(a)}{=} \sum_{h \in \mathcal{H}} B(h|sa) \sum_{s', r'} \sum_{o': \phi(h')=s'} \mu(o' r' | ha) f(s', r') \\ & \stackrel{(b)}{=} \sum_{h \in \mathcal{H}} B(h|sa) \sum_{s', r'} \mu_\phi(s' r' | ha) f(s', r') \\ & \stackrel{(c)}{=} \sum_{s', r'} \bar{\mu}(s' r' | sa) f(s', r') \end{aligned}$$

For (a) we sum over all o' by first summing over all o' such that $\phi(hao' r') = s'$, then summing over all s' . In (b) we used Definition 14.2.1. In (c) we used Definition 14.2.3. ■

Lemma 14.2.5: Dispersion Probability Equivalence III

Additionally, with the dispersion probability B we define the B -**average** of a Q-value for all histories $\tilde{h}$ that ϕ maps to the same state as h :

$$\langle Q(h, a) \rangle_B := \sum_{\tilde{h} \in \mathcal{H}} B(\tilde{h} | \phi(h) a) Q(\tilde{h}, a) \quad \text{where } s = \phi(h) \quad (14.2.6)$$

Lemma 14.2.7: B -Average Bounds I

Lemma 14.2.7 (B -Average Bounds)

For any $\mu, \phi, B, \pi, \bar{\pi}$, define $\bar{\mu}$ via Definitions 14.2.1 and 14.2.3. Then

$$\text{If } |V_{\mu}^{\pi}(h) - V_{\bar{\mu}}^{\bar{\pi}}(s)| \leq \delta \quad \forall s = \phi(h) \quad \text{then} \quad |\langle Q_{\mu}^{\pi}(h, a) \rangle_B - Q_{\bar{\mu}}^{\bar{\pi}}(s, a)| \leq \gamma \delta \quad \forall s = \phi(h), a$$

$$\text{If } |V_{\mu}^{*}(h) - V_{\bar{\mu}}^{*}(s)| \leq \delta \quad \forall s = \phi(h) \quad \text{then} \quad |\langle Q_{\mu}^{*}(h, a) \rangle_B - Q_{\bar{\mu}}^{*}(s, a)| \leq \gamma \delta \quad \forall s = \phi(h), a$$

Plain English. If the true value function V_{μ}^{π} (on histories) and the surrogate value function $V_{\bar{\mu}}^{\bar{\pi}}$ (on states) are close for all histories mapping to a given state, then their corresponding B -averaged Q -values are close too, up to an extra factor of γ (which only shrinks the gap, since $\gamma < 1$). The same holds for the optimal (starred) value/ Q -functions.

Lemma 14.2.7: B -Average Bounds II

Proof. Let $s := \phi(h)$, $s' := \phi(h')$, $h' = hao'r'$.

$$\begin{aligned}
 \langle Q_{\mu}^{\pi}(h, a) \rangle_B &\stackrel{(a)}{=} \sum_{\tilde{h} \in \mathcal{H}} B(\tilde{h} | \phi(h) a) Q_{\mu}^{\pi}(\tilde{h}, a) \\
 &\stackrel{(b)}{=} \sum_{\tilde{h} \in \mathcal{H}} B(\tilde{h} | \phi(h) a) \sum_{o', r'} \mu(o' r' | \tilde{h} a) (r' + \gamma V_{\mu}^{\pi}(\tilde{h}')) \\
 &\stackrel{(c)}{\leq} \sum_{\tilde{h} \in \mathcal{H}} B(\tilde{h} | \phi(h) a) \sum_{o', r'} \mu(o' r' | \tilde{h} a) (r' + \gamma (V_{\mu}^{\bar{\pi}}(s') \pm \delta)) \\
 &\stackrel{(d)}{=} \sum_{s', r'} \bar{\mu}(s' r' | sa) (r' + \gamma V_{\mu}^{\bar{\pi}}(s')) + \gamma \delta \\
 &\stackrel{(e)}{=} Q_{\mu}^{\bar{\pi}}(s, a) \pm \gamma \delta
 \end{aligned}$$

(a) is (14.2.6). (b) is the Bellman equation for Q -values. (c) is from the lemma's assumptions ($c \leq a \pm b$ means $c \leq a + b$ and $c \geq a - b$). (d) comes from Lemma 14.2.5. (e) is from (14.1.1).

Lemma 14.2.7: B -Average Bounds III

Lower and upper bounds together give $|\langle Q_\mu^\pi(h, a) \rangle_B - Q_{\bar{\mu}}^{\bar{\pi}}(s, a)| \leq \gamma\delta$. The second result follows by replacing π with π_μ^* and $\bar{\pi}$ with $\bar{\pi}_{\bar{\mu}}^*$; note that in general $\pi_\mu^* \neq \bar{\pi}_{\bar{\mu}}^*$. ■

Theorem 14.2.8: Q-Value Inheritance

Theorem 14.2.8 (Q-Value Inheritance [Hut16, Thm.5])

For any μ, ϕ, B , define $\bar{\mu}$ via Definitions 14.2.1 and 14.2.3. Let π be a policy such that $|Q_\mu^\pi(h, a) - Q_\mu^\pi(\tilde{h}, a)| \leq \varepsilon_q$ and either $\pi(h) = \pi(\tilde{h})$ with $\varepsilon_v = 0$, or $|V_\mu^\pi(h) - V_\mu^\pi(\tilde{h})| \leq \varepsilon_v$, for all $\phi(h) = \phi(\tilde{h})$ and all a . Then for all a and h :

$$|Q_\mu^\pi(h, a) - Q_{\bar{\mu}}^\pi(s, a)| \leq \frac{\varepsilon}{1 - \gamma} \quad \text{and} \quad |V_\mu^\pi(h) - V_{\bar{\mu}}^\pi(s)| \leq \frac{\varepsilon}{1 - \gamma}$$

where $\bar{\pi}(s) = \pi(h)$, $s = \phi(h)$, and $\varepsilon = \varepsilon_q + \varepsilon_v$.

Plain English. If two histories $h, \tilde{h}$ map to the same state and have close Q-values (within ε_q) and either the *same* action under π or close values (within ε_v), then the true Q-value/value on histories is close to the surrogate MDP's Q-value/value on states – within $\varepsilon/(1 - \gamma)$. So being careful to aggregate histories with similar behavior is enough to “inherit” good performance from the small MDP.

Proof of Theorem 14.2.8 I

Proof. Let

$$\delta := \sup_{s=\phi(h), a} |Q_{\mu}^{\pi}(h, a) - Q_{\mu}^{\bar{\pi}}(s, a)| \quad (14.2.9)$$

Step 1. We show $|V_{\mu}^{\pi}(h) - V_{\mu}^{\bar{\pi}}(s)| \leq \delta + \varepsilon_v \forall s = \phi(h)$ (Eq. 14.2.10). Under either condition of the theorem this follows from (6.7.4) and (14.1.1) and the definition of the supremum. For the or-condition, let $\bar{\pi}(s) := \pi(\tilde{h})$ for some $\tilde{h} \in \phi^{-1}(s)$. Then

$$|V_{\mu}^{\pi}(h) - V_{\mu}^{\bar{\pi}}(s)| \stackrel{(a)}{\leq} |V_{\mu}^{\pi}(\tilde{h}) - V_{\mu}^{\bar{\pi}}(s)| + \varepsilon_v \stackrel{(b)}{=} |Q_{\mu}^{\pi}(\tilde{h}, \tilde{a}) - Q_{\mu}^{\bar{\pi}}(s, \tilde{a})| + \varepsilon_v \stackrel{(c)}{\leq} \delta + \varepsilon_v$$

where (a) is the assumptions plus the triangle inequality, (b) uses (6.7.4) and (14.1.1) with $\tilde{a} = \bar{\pi}(s) = \pi(\tilde{h})$, and (c) is the definition of δ .

Proof of Theorem 14.2.8 II

Step 2. By Lemma 14.2.7, (14.2.10) implies

$$|\langle Q_\mu^\pi(h, a) \rangle_B - Q_\mu^{\bar{\pi}}(s, a)| \leq \gamma(\delta + \varepsilon_\nu) \quad \forall s = \phi(h) \quad (14.2.11)$$

By the assumption on Q_μ^π and B , for $s = \phi(h)$ we have (using $c \leq a \pm b$ notation again)

$$\langle Q_\mu^\pi(h, a) \rangle_B = \sum_{\tilde{h}: \phi(\tilde{h})=s} B(\tilde{h}|ha) Q_\mu^\pi(\tilde{h}, a) \leq \sum_{\tilde{h}: \phi(\tilde{h})=s} B(\tilde{h}|ha) (Q_\mu^\pi(h, a) \pm \varepsilon_q) = Q_\mu^\pi(h, a) \pm \varepsilon_q$$

Together with (14.2.11) this implies $|Q_\mu^\pi(h, a) - Q_\mu^{\bar{\pi}}(s, a)| \leq \gamma(\delta + \varepsilon_\nu) + \varepsilon_q$, hence $\delta \leq \gamma(\delta + \varepsilon_\nu) + \varepsilon_q$, and rearranging gives $\delta \leq \frac{\varepsilon_q + \gamma\varepsilon_\nu}{1-\gamma}$. Inserting this bound into (14.2.9) and (14.2.10) gives the bounds in the theorem. ■

Theorem 14.2.12: Value Inheritance

Theorem 14.2.12 (Value Inheritance [Hut16, Thm.6])

For any μ, ϕ, B , define $\bar{\mu}$ via Definitions 14.2.1 and 14.2.3. Let π be a policy such that $|V_{\mu}^{\pi}(h) - V_{\mu}^{\pi}(\tilde{h})| \leq \varepsilon$ for all $\phi(h) = \phi(\tilde{h})$. Then for all a and h :

$$|V_{\mu}^{\pi}(h) - V_{\bar{\mu}}^{\bar{\pi}}(s)| \leq \frac{\varepsilon}{1 - \gamma} \quad \text{and} \quad |Q_{\bar{\mu}}^{\bar{\pi}}(s, a) - \langle Q_{\mu}^{\pi}(h, a) \rangle_B| \leq \frac{\varepsilon \gamma}{1 - \gamma}$$

where $\bar{\pi}(s) = \pi(h)$ and $s = \phi(h)$.

Plain English. A companion result to Theorem 14.2.8: this time we only need the *value* function (not Q-values) to be close on aggregated histories, and we get a similar guarantee – the “closeness” of aggregated states is inherited by the MDP’s value function $V_{\bar{\mu}}^{\bar{\pi}}$.

Example 14.2.13: Aggregation to a Non-MDP

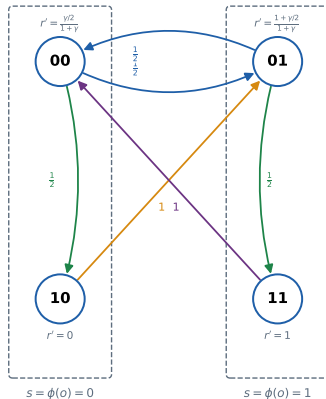
Consider a process μ that is itself an MDP in the observations, with transition matrix T and reward function R , i.e. $\mu(o'r'|ha) = T_{oo'}^a R_{oo'}^{ar'}$, where o is the last observation in h . Concretely, with observation space $\mathcal{O} = \{00, 01, 10, 11\}$:

$$T = \begin{pmatrix} \cdot & \frac{1}{2} & \frac{1}{2} & \cdot \\ \frac{1}{2} & \cdot & \cdot & \frac{1}{2} \\ \cdot & 1 & \cdot & \cdot \\ 1 & \cdot & \cdot & \cdot \end{pmatrix}, \quad R = \begin{pmatrix} \gamma/2/(1+\gamma) \\ (1+\gamma/2)/(1+\gamma) \\ 0 \\ 1 \end{pmatrix}$$

This is an action-independent Markov process with a deterministic reward R (i.e. $\mu(o'r'|ha) = T_{oo'} \cdot \mathbb{I}[r' = R(o)]$, with $\mathbb{I}[\cdot]$ the indicator function).

Example 14.2.13, Pictured

Example 14.2.13: reducing 4 observations to 2 states



Each circle is a raw observation $o \in \{00, 01, 10, 11\}$; arrows show transition probabilities from T ; r' labels show the deterministic reward $R(o)$ for leaving that observation. Dashed boxes show the 2-state aggregation we consider next.

Example 14.2.13, Continued: The Reduction Is Not Markov

Consider the reduction

$$s_t := \phi(h_t) := \begin{cases} 0 & \text{if } o_t = 00 \text{ or } 10 \\ 1 & \text{if } o_t = 01 \text{ or } 11 \end{cases} \in \mathcal{S} := \{0, 1\}$$

The reduced process μ_ϕ is **not** (even approximately) Markov:

$$\mu_\phi(s' = 0 | o = 00) = T_{00,00} + T_{00,10} = 0 + \frac{1}{2} = \frac{1}{2} \neq \mu_\phi(s' = 0 | o = 10) = T_{10,00} + T_{10,10} = 0 + 0 = 0$$

Plain English. Being told only “ $s = 0$ ” is not enough to predict the next state: it matters whether the underlying observation was 00 or 10, and those give *different* next-state probabilities. So μ violates the **bisimulation condition** [GDG03], and raw observations 00, 10 have a large “bisimulation distance” [FPP04, Ort07].

Yet the (Q-)Values agree. One can verify directly that $V(o_t) := V^\pi(h_t) = Q^\pi(h_t, a_t) \forall a_t$ satisfies

$$V(00) = V(10) = \frac{\gamma}{1 - \gamma^2} \quad \text{and} \quad V(01) = V(11) = \frac{1}{1 - \gamma^2}$$

That is, V and Q are ϕ -**uniform**: they agree exactly on histories ϕ maps together. The conditions of Theorems 14.2.8 and 14.2.12 hold exactly ($\varepsilon = 0$), so all four raw observations can be aggregated into just **two** states, *despite* $\mu_\phi \notin \text{MDP}$ (the policy is irrelevant here and can be chosen constant). ♦

Python: Reproducing Example 14.2.13

We rebuild T and R exactly, verify $\mu_\phi(s' = 0|o)$ against the book's values, then solve $V(o) = R(o) + \gamma \sum_{o'} T_{oo'} V(o')$ directly (recall $R(o)$ attaches to the *current* observation, per $\mu(o'r'|ha) = T_{oo'} \cdot \mathbb{I}[r' = R(o)]$) and compare to the closed forms.

```
1 import numpy as np
2 gamma = 0.9
3 obs = ["00", "01", "10", "11"]
4 T = np.array([[0, .5, .5, 0], [.5, 0, 0, .5],
5               [0, 1, 0, 0], [1, 0, 0, 0]])
6 Rvec = np.array([(gamma/2)/(1+gamma), (1+gamma/2)/(1+gamma), 0.0, 1.0])
7 phi = lambda o: 0 if o in ("00", "10") else 1
8
9 for i, o in enumerate(obs):
10     p0 = sum(T[i, obs.index(op)] for op in obs if phi(op) == 0)
11     print(f"mu_phi(s'=0|o={o}) = {p0}")
12
13 Vsol = np.linalg.solve(np.eye(4) - gamma * T, Rvec)    # V = R + gamma*T@V
14
15 # Output:
16 # mu_phi(s'=0|o=00) = 0.5    mu_phi(s'=0|o=01) = 0.5
17 # mu_phi(s'=0|o=10) = 0.0    mu_phi(s'=0|o=11) = 1.0
18 # solved V(00)=4.736842 V(01)=5.263158 V(10)=4.736842 V(11)=5.263158
19 # matches gamma/(1-gamma^2)=4.736842 and 1/(1-gamma^2)=5.263158 exactly.
```

Theorem 14.2.14: Optimal Q-Value Inheritance

Theorem 14.2.14 (Optimal Q-Value Inheritance [Hut16, Thm.8])

For any μ, ϕ, B , define $\bar{\mu}$. Assume $|Q_\mu^*(h, a) - Q_\mu^*(\tilde{h}, a)| \leq \varepsilon$ for all $\phi(h) = \phi(\tilde{h})$ and all a . Then for all a, h , and $s = \phi(h)$:

- $|Q_\mu^*(h, a) - Q_{\bar{\mu}}^*(s, a)| \leq \frac{\varepsilon}{1-\gamma}$ and $|V_\mu^*(h) - V_{\bar{\mu}}^*(s)| \leq \frac{\varepsilon}{1-\gamma}$
- $0 \leq Q_\mu^*(h, a) - Q_{\bar{\mu}}^{\tilde{\pi}}(h, a) \leq \frac{2\varepsilon\gamma}{(1-\gamma)^2}$ and $0 \leq V_\mu^*(h) - V_{\bar{\mu}}^{\tilde{\pi}}(h) \leq \frac{2\varepsilon\gamma}{(1-\gamma)^2}$
- If $\varepsilon = 0$, then $\pi^*(h) = \bar{\pi}_{\bar{\mu}}^*(s)$

where $\tilde{\pi}(h) := \bar{\pi}_{\bar{\mu}}^*(s)$.

Plain English. If histories that ϕ maps together have close *optimal* Q-values, then the optimal Q-values/values of $\bar{\mu}$ (on states) are close to those of μ (on histories), *and* the “uplifted” policy $\tilde{\pi}$ (act according to the small MDP’s optimal policy) is itself close to optimal in the original problem. When $\varepsilon = 0$ exactly, the uplifted policy *is* the optimal policy.

Why This Theorem Matters, and a Caveat

This theorem shows that when ϕ aggregates histories with close optimal Q-values, we can aggregate histories *as much as we wish*, as long as the optimal Q-value function and policy remain approximately representable as functions of the aggregated states. Whether the reduced process μ_ϕ is Markov or not is **immaterial**. We can use the surrogate MDP $\bar{\mu}$ to find an ε -optimal policy for μ – this will prove useful when constructing a map ϕ that satisfies the theorem's assumptions.

Remark 14.2.15 (No Optimal Value Inheritance)

The Q-value bound generalizes naturally from Theorem 14.2.12 to the optimal case, but somewhat surprisingly the *value* bound does **not** generalize to optimal policies [Hut16, Thm.10].

Definition 14.2.16: Extreme State Aggregation

We now give an example of an extreme state aggregation map ϕ that satisfies the premises of Theorem 14.2.14.

Definition 14.2.16 (Extreme State Aggregation)

Consider ϕ^{ESA} that maps each history to the **vector-over-actions of optimal Q-values** $Q_\mu^*(h, \cdot)$, discretized to some finite ε -grid:

$$\phi^{ESA}(h) := \left(\left\lfloor \frac{Q_\mu^*(h, a)}{\varepsilon} \right\rfloor \right)_{a \in \mathcal{A}} \in \left\{ 0, \dots, \left\lfloor \frac{1}{\varepsilon(1-\gamma)} \right\rfloor \right\}^{\mathcal{A}} =: \mathcal{S}$$

Plain English. $\lfloor \cdot \rfloor$ is the **floor** (round-down) function. For *each* action a , we take the true optimal Q-value $Q_\mu^*(h, a)$, divide it by a small grid spacing ε , and round down – like snapping a real number onto a grid. The state is the resulting tuple of grid cells, one entry per action. Histories with ε -close Q_μ^* -values in *every* action land in exactly the same grid cell, hence the same state – this is obvious from the construction.

Given μ, B, ϕ we build $\bar{\mu}$, find its optimal policy $\bar{\pi}_{\bar{\mu}}^*$, and *uplift* it via $\tilde{\pi}(h) := \bar{\pi}_{\bar{\mu}}^*(\phi(h))$. By Theorem 14.2.14, $\tilde{\pi}(h)$ is ε' -optimal in μ , where $\varepsilon' := 2\varepsilon(1-\gamma)^2$ (when $\varepsilon = 0$, Q_μ^* is bounded by $\frac{1}{1-\gamma}$).

Theorem 14.2.17: Extreme ϕ and Its Size Bound I

Theorem 14.2.17 (Extreme ϕ)

For every process μ , there exists a feature map ϕ (Definition 14.2.16), namely ϕ^{ESA} , and an MDP $\bar{\mu}$ (Definitions 14.2.1, 14.2.3), whose optimal policy $\bar{\pi}^*$ is an ε' -optimal policy $\tilde{\pi}(h) := \bar{\pi}^*(\phi(h))$ for μ . The size of the MDP $\bar{\mu}$ is bounded (uniformly for any μ) by

$$|\mathcal{S}| \leq \left(\frac{3}{\varepsilon'(1-\gamma)^3} \right)^{|\mathcal{A}|}$$

Plain English. No matter how complicated μ is, there is *always* some finite-state MDP – built purely from the shape of Q_μ^* – whose optimal policy, once “uplifted” back to histories, performs within ε' of optimal. And its size does not depend on the observation or reward space at all, only on ε' , γ , and the number of actions $|\mathcal{A}|$.

Theorem 14.2.17: Extreme ϕ and Its Size Bound II

Proof. If $\varepsilon' > \frac{1}{1-\gamma}$ then *any* policy is ε' -optimal (since $Q_\mu^* \leq \frac{1}{1-\gamma} < \varepsilon'$), so the result holds trivially by choosing a trivial state class $|\mathcal{S}| = 1$. In the case $\varepsilon' \leq \frac{1}{1-\gamma}$, from the definition of $\mathcal{S}$ in Definition 14.2.16 we have

$$|\mathcal{S}| = \left(\left\lfloor \frac{1}{\varepsilon(1-\gamma)} \right\rfloor + 1 \right)^{|\mathcal{A}|} = \left(\left\lfloor \frac{2}{\varepsilon'(1-\gamma)^3} \right\rfloor + 1 \right)^{|\mathcal{A}|} \leq \left(\frac{3}{\varepsilon'(1-\gamma)^3} \right)^{|\mathcal{A}|} \quad \blacksquare$$

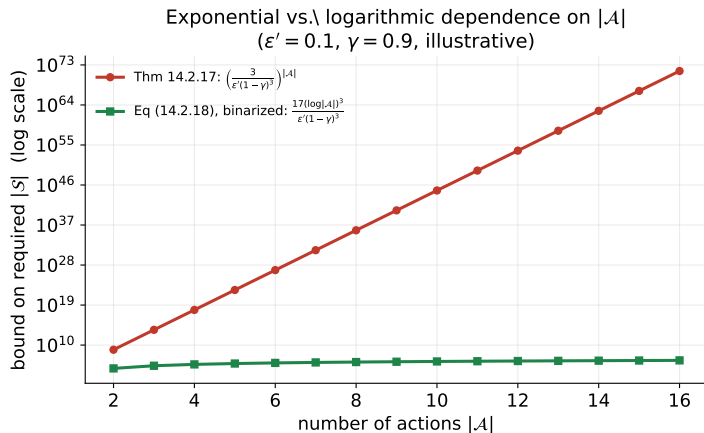
Theorem 14.2.17: Extreme ϕ and Its Size Bound III

Interestingly, these bounds do **not** depend on the size of the observation space, the reward space, or the complexity of μ – because extreme state aggregation only cares about aggregating histories with similar (Q-)Values, regardless of what the original spaces or the resulting Markov property look like.

Unfortunately the bound is *exponential* in the action space size, and due to the high powers of ε' and $1 - \gamma$ can be very large. But it was shown in [MH21b,MH21a] that if the action space is *sequentialized* into binary actions, the bound improves from exponential to *logarithmic* in $|\mathcal{A}|$ – a double-exponential improvement:

$$|\mathcal{S}| \leq \frac{17(\log |\mathcal{A}|)^3}{\varepsilon'(1 - \gamma)^3} \quad (14.2.18)$$

Theorem 14.2.17: Extreme ϕ and Its Size Bound IV



Section 14.2.4: Feature Reinforcement Learning in Practice

In practice, we do not know Q_μ^* (which would imply the optimal policy is already known), so we do not have access to the extreme ϕ -map of Definition 14.2.16. That map is *not* meant for practical use – it exists to demonstrate that a ϕ -map satisfying our theorems' premises exists at all.

Reinforcement learning *learns* Q_μ^* from samples. Taking a model-based approach, we can first learn $\bar{\mu}$, then use (14.1.2) to compute $Q_{\bar{\mu}}^*$. It turns out that for a very particular choice of B , we can follow *any* convenient behavior policy (off-policy learning), and a simple frequency estimate – counting (s, a, r', s') -pairs in the reduced history $\bar{h}$ (see Section 14.3.1) – indeed estimates $\bar{\mu}$, despite being a surrogate MDP only related to the true sampling distribution μ via Definitions 14.2.1 and 14.2.3.

The chicken-egg problem. All this still assumed ϕ is given. Ultimately the goal of FRL is to also *learn* a map ϕ satisfying the premises – but this again requires first finding Q_μ^* , and thus defeats the purpose. This is a classical, but solvable, chicken-egg problem: algorithms to learn ϕ were proposed in [Hut16] and developed in [Maj21].

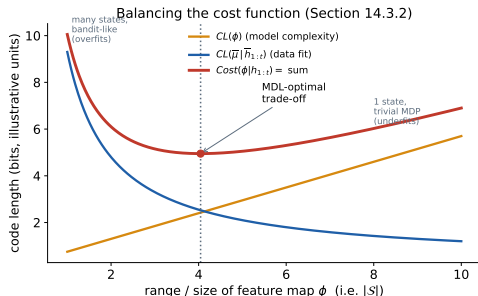
Outline

- 1 Introduction: An Alternative to AIXI
- 2 14.1 Feature Reinforcement Learning Setup
- 3 14.2 History Aggregation beyond MDPs
 - 14.2.1 Surrogate MDP and Dispersion Probability
 - 14.2.2 (Q-)Value Inheritance for Fixed and Optimal Policy
 - 14.2.3 Extreme State Aggregation
- 4 14.3 Feature MDP
 - 14.3.1 Feature Learning
 - 14.3.2 Choices of the Cost Function
 - 14.3.3 Feature Dynamic Bayesian Networks
- 5 14.4 Context Tree Maximization Reinforcement Learning
- 6 14.5 Exercises
- 7 14.6 History and References

Feature MDP (Φ MDP): A Practical Construction

In the previous section we discussed ideal properties for ϕ (aggregating close Q-values), then used that criterion to construct an explicit (but impractical) ϕ . This section provides a **practical** method for iteratively constructing ϕ , based on a **cost function** over choices of ϕ [Hut09f,Das16].

A crucial difference: here we *require* the aggregation to be approximately an MDP. We want ϕ to map to a small state space, yet large enough that the resulting state dynamics is (still approximately) an MDP. So the cost function must balance ϕ 's with small range against ϕ 's that lead to good (approximate) MDPs, and should be minimal for the “best” compromise – an idea in the spirit of the **MDL principle**.



Definition 14.3.1: Feature Map Cost

Definition 14.3.1 (Feature Map Cost)

The *Cost* of $\phi : \mathcal{H} \rightarrow \mathcal{S}$ on $h_{1:t}$ is a measure of the quality of the feature map ϕ given history $h_{1:t}$.

Plain English. We delay the exact choice of *Cost* until Section 14.3.2. Ideally $\arg \min_{\phi} \text{Cost}(\phi | h_{1:t})$ could be computed exactly, but for most interesting/useful choices of *Cost* this is intractable, so we turn to iterative methods instead.

Suffix feature maps. For concreteness, we consider the class of feature maps $\{\phi_{\mathcal{S}}\}$ that take the *most recent observations* as state: the state space forms a **suffix set** $\mathcal{S} \subset \mathcal{O}^*$, and $\phi_{\mathcal{S}}(h_{1:t}) = o_{k:t}$ for the unique k such that $o_{k:t} \in \mathcal{S}$. (We can generalize to non-binary alphabets or binarize $\mathcal{O}$ first and assume $\mathcal{S} \subset \mathbb{B}^*$.) We write $s_{>1}$ to denote s without its first element.

What Φ Improve Does

Φ Improve modifies a state space S by either **splitting** one state into several new states, or **merging** several states (if capable of merging) into one – in suffix-tree jargon, expanding or shrinking the suffix tree at some leaf nodes. If the resulting ϕ' has lower *Cost*, the algorithm outputs ϕ' ; otherwise it outputs the unchanged ϕ (but sometimes accepts a *slightly worse* ϕ' anyway, to escape local minima – via the $\log(q)$ random-acceptance test on the algorithm's last line). The pseudocode is on the next slide.

Algorithm 14.1: Φ Improve

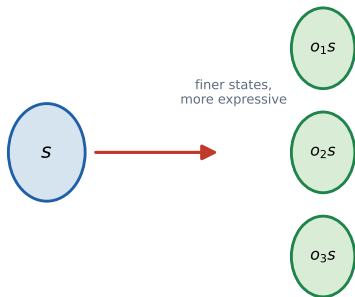
Input: State space $\mathcal{S}$; feature map $\phi_{\mathcal{S}} : \mathcal{H} \rightarrow \mathcal{S}$; history $h_{<t}$

Output: Improved feature map $\phi_{\mathcal{S}'} : \mathcal{H} \rightarrow \mathcal{S}'$

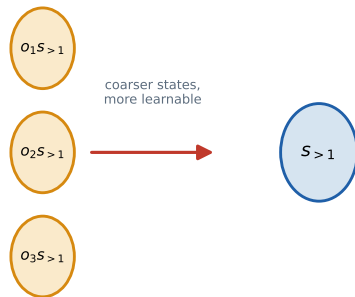
- 1: Randomly choose a state $s \in \mathcal{S}$
- 2: Let p, q be uniform random numbers in $[0, 1]$
- 3: $\mathcal{S}' := \mathcal{S}$; $\phi_{\mathcal{S}'} := \phi_{\mathcal{S}}$
- 4: **if** $p > 1/2$ **then**
 - 5: $\mathcal{S}' := (\mathcal{S} \setminus \{s\}) \cup \{os : o \in \mathcal{O}\}$ ▷ Split $\{s\}$ into $\{os : o \in \mathcal{O}\}$
 - 6: $\phi_{\mathcal{S}'}(h) \in \{os : o \in \mathcal{O}\}$ for all h s.t. $\phi_{\mathcal{S}}(h) = s$ ▷ chosen by some rule, e.g. randomly
- 7: **else if** $\{os_{>1} : o \in \mathcal{O}\} \subseteq \mathcal{S}$ **then**
 - 8: $\mathcal{S}' := (\mathcal{S} \setminus \{os_{>1} : o \in \mathcal{O}\}) \cup \{s_{>1}\}$ ▷ Merge $\{os_{>1} : o \in \mathcal{O}\}$ into $\{s_{>1}\}$
 - 9: $\phi_{\mathcal{S}'}(h) := s_{>1}$ for all h such that $\phi_{\mathcal{S}}(h) \in \{os_{>1} : o \in \mathcal{O}\}$
- 10: **end if**
- 11: **if** $Cost(\phi_{\mathcal{S}}, h_{<t}) - Cost(\phi_{\mathcal{S}'}, h_{<t}) > \log(q)$ **then**
- 12: **return** $\phi_{\mathcal{S}'}$
- 13: **else**
- 14: **return** $\phi_{\mathcal{S}}$
- 15: **end if**

Split and Merge, Pictured

Split: $\{s\} \rightarrow \{os : o \in \mathcal{O}\}$



Merge: $\{os_{>1} : o \in \mathcal{O}\} \rightarrow \{s_{>1}\}$



Split increases the resolution of the state space (more, finer states) so that the resulting dynamics can capture more structure. **Merge** does the opposite: it coarsens the state space, trading some expressiveness for a smaller, more easily learned MDP. Φ Improve proposes one such move at random and keeps it only if the total *Cost* improves (or, occasionally, even if it does not – to avoid getting stuck).

Building the MDP: Frequency Estimation

Given a feature map ϕ we construct the state at each time step as ϕ of the history, e.g. $s_t = \phi(h_{1:t})$. We can then build a **state history** $\bar{h}_{1:t} := a_1 r_1 s_1 \dots a_t r_t s_t$. Given the state history, we *approximate* the dynamics of the MDP through **frequency estimation** on the transitions of the MDP:

Frequency-Estimate MDP – Eq. (14.3.2)

$$\bar{\mu}(s' r' | s, a) := \begin{cases} \frac{N(s a r' s', \bar{h}_{1:t})}{N(s a, \bar{h}_{1:t})} & \text{if } N(s a, \bar{h}_{1:t}) > 0 \\ 0 & \text{otherwise} \end{cases}$$

Plain English. $N(s a r' s', \bar{h}_{1:t})$ counts how many times the transition $(s, a) \rightarrow (s', r')$ has occurred in the state history so far, and $N(s a, \bar{h}_{1:t})$ counts how many times (s, a) occurred at all. Dividing gives the empirical (observed) probability of each transition – the most natural estimate one could use. (Strictly this is only an *estimate* $\hat{\bar{\mu}}$ of $\bar{\mu}$.)

Algorithm 14.2: The Φ MDP Agent

Require: Action Space $\mathcal{A}$; Percept Space $\mathcal{E} = \mathcal{O} \times \mathcal{R}$

Input: Percept stream $e_1, e_2, e_3, \dots$

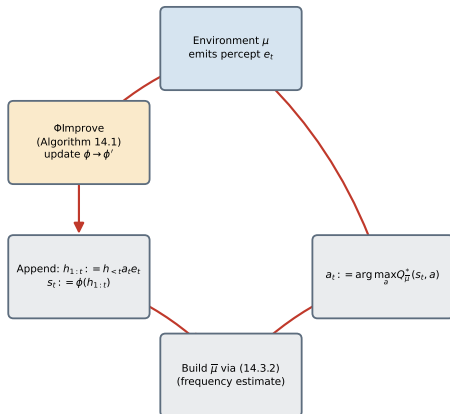
Output: Action stream $a_1, a_2, a_3, \dots$

```
1:  $\mathcal{S} := \{\epsilon\}; \quad \phi(h) := \epsilon, \phi'(h) := \epsilon \forall h; \quad h_0, \bar{h}_0, a_0, e_0 := \epsilon$ 
2: for  $t = 1, \dots$  do
3:    $\phi' := \phi$ 
4:   while waiting for next  $e_t$  do
5:      $\phi' := \Phi\text{Improve}(\mathcal{S}, \phi', h_{<t})$ 
6:     if  $\text{Cost}(\phi' | h_{<t}) < \text{Cost}(\phi | h_{<t})$  then
7:        $\phi := \phi'$ 
8:     end if
9:   end while
10:  Receive  $e_t$  from environment  $\mu$ 
11:   $h_{1:t} := h_{<t} a_t e_t; \quad s_t := \phi(h_{1:t}); \quad \bar{h}_{1:t} := \bar{h}_{<t} a_t r_t s_t; \quad \mathcal{S} := \mathcal{S} \cup \{s_t\}$ 
12:  Construct  $\bar{\mu}$  by (14.3.2)
13:   $a_t := \arg \max_a Q_{\bar{\mu}}^*(s_t, a); \quad \text{output action } a_t$ 
14: end for
```

What Algorithm 14.2 Does

While waiting for the next percept, the agent keeps proposing improved feature maps via $\Phi\text{Improve}$ and switches to them if they are cheaper. Once the percept arrives, it updates its history/state, rebuilds the MDP via frequency estimation, and acts greedily according to the resulting $Q_{\bar{\mu}}^*$.

Algorithm 14.2: the ΦMDP agent loop



loop for $t = 1, 2, 3, \dots$ (output action a_t , receive next percept e_{t+1})

A Note on Exploration

To ensure sufficient exploration, we can set the reward to be **high** for unexplored state-action interactions when estimating Q_{μ}^* – an “optimism” trick reminiscent of Chapter 9. We can solve Q_{μ}^* efficiently with policy iteration, value iteration, or another polynomial-time method. Note that by construction, the agent has always *visited* every state, but it has not necessarily *taken every action* in every state.

Using Φ MDP we are able to solve the hard problem of general/history-based reinforcement learning by mapping the problem to the easier MDP setting, for which we have efficient solutions.

Balancing Two Extremes

Since we are interested in choices of ϕ that let the resulting Φ MDP agent perform well, we want ϕ to result in states that capture the dynamics of the underlying environment μ sufficiently well. Many immediate questions arise, so let's examine two extremes:

- We could choose a ϕ that maps every unique history to a *different* state. This fully captures the dynamics of μ , but we are also interested in *solving* the resulting MDP – and with this choice, the resulting MDP is exactly as hard as the original problem.
- We could instead choose a ϕ that maps *all* histories to a single state. The resulting MDP (now a bandit problem) has many effective algorithmic solutions – but since it does not capture the correct dynamics of μ , it will likely not perform well.

We need a balance between these two extremes. To this end we turn to **simplicity** (as we often do in this book) as a guiding principle – surely both extreme choices of ϕ above are simple, if we are only interested in *states*. We are interested in using *both* the states and rewards of our resulting MDP; however, our ϕ is only responsible for the states – the rewards the Φ MDP agent is given are the true rewards from the environment μ .

The MDL-Inspired Cost Function

We take inspiration from the **MDL principle** (minimum description length) and measure the simplicity of a mapping ϕ given a history $h_{1:t}$ as the size of an encoding of $h_{1:t}$ with ϕ , plus the size of encoding ϕ itself. By “an encoding of $h_{1:t}$ given ϕ ” we mean an encoding of the resulting $\bar{\mu}$ given ϕ . Because $\bar{\mu}$ is based on the frequencies of state/reward transitions, we can encode the transitions, using the size of that (partial) encoding of $h_{1:t}$ with ϕ as our cost.

Cost Function – Eq. (14.3.3)

$$\text{Cost}(\phi|h_{1:t}) := CL(\bar{\mu}|\bar{h}_{1:t}) + CL(\phi)$$

Plain English. CL denotes the **code length** – the number of bits needed to describe something, using the best code available. So Cost has two terms: how many bits it takes to describe the observed transitions using $\bar{\mu}$ (goodness of fit), plus how many bits it takes to describe ϕ itself (model complexity) – precisely the MDL trade-off pictured a few slides ago.

Since $\bar{\mu}(s'r'|sa)$ decomposes as $\bar{\mu}(s'|sa)\bar{\mu}(r'|sa, s')$, and these two factors are independent, the information of the two together is the sum of the information of each:

$$\text{Cost}(\phi|h_{1:t}) := CL(\bar{\mu}(s'|sa)|\bar{h}_{1:t}) + CL(\bar{\mu}(r'|sa, s')|\bar{h}_{1:t}) + CL(\phi)$$

Coding Schemes for $\bar{\mu}$

To find $CL(\bar{\mu}(s'|sa)|\bar{h}_{1:t})$ we are interested in the **counts** of each new state s' given the previous state and action s, a , and similarly the counts of new rewards r' given s, a, s' . There are multiple coding schemes that can be used here:

- the **Minimal Description Length** code,
- the **Combinatorial** code,
- the **Incremental** code, and
- the **Bayesian** code.

We will not specify a particular coding algorithm here – any consistent choice completes the description of $\Phi\text{Improve}(\mathcal{S}, \phi_{\mathcal{S}}, h_{<t}, \text{Algorithm 14.1})$ and hence this instantiation of the overall ΦMDP agent (Algorithm 14.2). Implementations and experimental results can be found in [Ngu13,Das16].

Φ DBN: Factored State Spaces

Feature Dynamic Bayesian Networks (Φ DBN) [Hut09d,Hut09a] are a special case of Φ MDP for structured, *factored* state spaces. Recall that in an MDP the transition probabilities depend only on the previous state and action; in a **Dynamic Bayesian Network**, the state space is *factorized* and the transition probabilities only depend on *specific components* of this factorization. These components are called **features**. DBNs are to MDPs what Bayesian networks are to a full tabular (unstructured) joint distribution – this structure/sparsity allows learning MDPs with much larger state spaces.

We specifically consider the case where the state space factorizes into m binary components: $\mathcal{S} = \mathbb{B}^m$. In Φ DBN, ϕ is a collection specifying which binary components of the *next* state depend on which binary components of the *previous* state. We use superscript to denote the bitwise components and subscript for the time component; e.g. s^i is the i -th feature of state s .

The Factorized Transition Structure

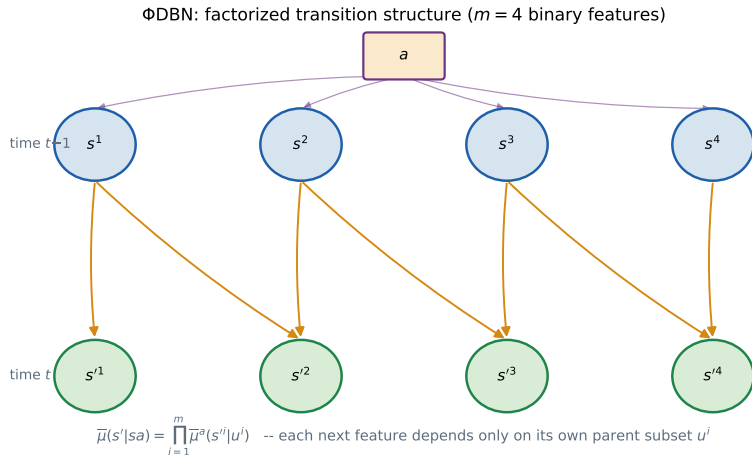
We assume the individual features of the next state $s' \in \mathcal{S}$ are independent of each other (s'^j and s'^i are independent for all i, j), and that each s'^i depends only on a subset of **parent** features $u^i \in \mathbb{B}^m$:

Φ DBN Factorization – Eq. (14.3.4)

$$\bar{\mu}(s'|sa) = \prod_{i=1}^m \bar{\mu}^a(s'^i|u^i)$$

Plain English. $\bar{\mu}^a$ is an unknown, action-dependent transition probability for each feature. Instead of one giant transition table over all of $\mathcal{S} = \mathbb{B}^m$, we get m small tables, one per feature, each conditioned only on that feature's parents u^i (a subset of the previous state's bits) – exactly the sparsity that makes DBNs tractable for large m .

The Φ DBN Factorization, Pictured



With $m = 4$ binary features, each next-feature node s'^i only has arrows coming in from its own small parent subset $u^i \subseteq \{s^1, \dots, s^m\}$ (orange) and the action a (purple) – not from every feature of the previous state. This sparsity is exactly what keeps Φ DBN tractable even when m is large.

The KT-Based Estimate

Since we do not know $\bar{\mu}^a$, we must estimate it. We could use frequency estimation as in Φ MDP above, but instead we use the following **KT-based** estimate (Definition 4.1.1), which regularizes the counts and works better for rarely visited transitions:

KT-Based Estimate for Φ DBN

$$\bar{\mu}^a(s^i|u^i) = \frac{|\{n \leq t : u_{n-1} = u^i, a_{n-1} = a, s_n^i = s^i\}| + 1/2}{|\{n \leq t : u_{n-1} = u^i, a_{n-1} = a\}| + 1}$$

Plain English. This is exactly the **Krichevsky-Trofimov (KT) estimator** from Chapter 4/Section 12.1: count how often feature i became s^i after seeing parents u^i and action a , add $\frac{1}{2}$ (a “pseudo-count” that avoids ever predicting probability 0 or 1 from finite data), and divide by the total count of that context plus 1.

Using this, we estimate $\bar{\mu}$ and the probability of a state sequence $s_{1:n}$ given actions $a_{1:n}$ by

$$\bar{\mu}(\bar{h}_{1:t}) = \prod_{n=1}^t \bar{\mu}(s_n | s_{n-1} a_{n-1}) = \prod_{n=1}^t \prod_{i=1}^m \frac{|\{n \leq t : u_{n-1} = u^i, a_{n-1} = a, s_n^i = s^i\}| + 1/2}{|\{n \leq t : u_{n-1} = u^i, a_{n-1} = a\}| + 1}$$

Python: KT Estimator for a Φ DBN Feature

We stream 8 samples of one binary feature s^i with parent $u^i = s_{t-1}^1$ (a single previous bit) under a constant action, and track how the KT estimate $\bar{\mu}^a(s' = 1|u)$ evolves.

```
1 from collections import defaultdict
2 data = [(0, "a", 1), (1, "a", 1), (1, "a", 0), (0, "a", 1),
3         (1, "a", 1), (0, "a", 1), (1, "a", 0), (1, "a", 1)]
4 count_u, count_u1 = defaultdict(int), defaultdict(int)
5
6 def kt(u, a):
7     n1, n = count_u1[(u,a)], count_u[(u,a)]
8     return (n1 + 0.5) / (n + 1)           # KT estimate, Eq. above
9
10 for u, a, sp in data:
11     pred = kt(u, a)                       # prediction BEFORE seeing sp
12     count_u[(u,a)] += 1
13     if sp == 1: count_u1[(u,a)] += 1
14
15 # Output (predictions made just before each new sample arrives):
16 # u=0: 0.5000 -> 0.7500 -> 0.8333 (observed s'=1 all three times)
17 # u=1: 0.5000 -> 0.7500 -> 0.5000 -> 0.6250 -> 0.5000
18 # Final estimates:
19 #    $\mu\text{-bar}^a(s'=1|u=0) = (3+1/2)/(3+1) = 0.8750$ 
20 #    $\mu\text{-bar}^a(s'=1|u=1) = (3+1/2)/(5+1) = 0.5833$ 
```

The estimate for $u = 0$ climbs toward 1 (all observed outcomes were 1); $u = 1$ stays near 0.5 (mixed outcomes) – the regularized, “never fully certain” behavior the $+\frac{1}{2}/+1$ counts produce.

Encoding Rewards in Φ DBN

For the encoding of rewards we assume the reward for a next state can be written as the sum of functions that each depend only on an individual feature:

$$R(s) := \sum_{i=1}^m R^i(s^i)$$

This is reasonable, since we are assuming each feature of the state is independent of the others, and that transitions to new states/rewards depend only on the previous state and action. Since features $s^i \in \{0, 1\}$, this is equivalent to a **linear function** of the feature vector $s \in \mathbb{B}^m$, for any choice of $R^i(\cdot)$:

$$R(s) := w_0 + w^\top s = w_0 + w_1 s^1 + \cdots + w_m s^m$$

for a suitable weight vector $w \in \mathbb{R}^{m+1}$. To determine if our weighting w is good, we need a loss function comparing $R(s_t)$ to r_t for all s_t, r_t in the history. **Least squares** is a sensible choice, though there are many possible choices; details can be found in [Hut09a]. Like in Section 14.3, we combine our code lengths of states and rewards to find the code length of a given ϕ :

$$Cost(\phi|h_{1:t}) := CL(\bar{\mu}(s'|s, a)|\bar{h}_{1:t}) + CL(\bar{\mu}(r|s, a, s')|\bar{h}_{1:t})$$

Outline

- 1 Introduction: An Alternative to AIXI
- 2 14.1 Feature Reinforcement Learning Setup
- 3 14.2 History Aggregation beyond MDPs
 - 14.2.1 Surrogate MDP and Dispersion Probability
 - 14.2.2 (Q-)Value Inheritance for Fixed and Optimal Policy
 - 14.2.3 Extreme State Aggregation
- 4 14.3 Feature MDP
 - 14.3.1 Feature Learning
 - 14.3.2 Choices of the Cost Function
 - 14.3.3 Feature Dynamic Bayesian Networks
- 5 14.4 Context Tree Maximization Reinforcement Learning
- 6 14.5 Exercises
- 7 14.6 History and References

CTMRL: FRL Meets Context Trees

Throughout this chapter we have gone over the basics of FRL but not yet demonstrated it in practice. **Context Tree Maximization Reinforcement Learning (CTMRL)** is to FRL what MC-AIXI-CTW is to AIXI. CTMRL [Ngu13,NSH12] finds the **maximal context tree** and uses it to construct an MDP, then takes the optimal action assuming the agent is in that MDP.

In Chapter 12 we saw how context trees emulate the dynamics of MDP environments, and that a Bayesian mixture over context trees can be done efficiently with **Context Tree Weighting (CTW)**: when the true environment was Markov, CTW performs well. In FRL we are *not* interested in mixing over Markov environments – instead we assume the true environment is **non-Markov** (completely history-dependent), but has some Markov encoding with an optimal feature map ϕ^* . In this sense context trees are still useful: our optimal feature map ϕ^* will describe an MDP, and that MDP can be encoded as a context tree. **Context Tree Maximization (CTM)** is a method to choose the optimal ϕ / context tree – recall CTM from Section 5.5 for binary sequence prediction; readers should familiarize themselves with that section first.

The CTM Cost Function

Let $\{\kappa_1, \dots, \kappa_{|\mathcal{A} \times \mathcal{E}|}\} = \mathcal{A} \times \mathcal{E}$ denote the ordered set of (action,percept) pairs. Given history $h_{1:t}$, let n_{κ_i} denote the number of occurrences of κ_i in that history. Let $P_e^{\kappa|sa}$ denote the block probability estimate of seeing a sequence containing n_{κ_i} occurrences of κ_i for all i , given state s and action a . Using $P_e^{\kappa|sa}$ we build the MDP $\bar{\mu}$ that produces a state history sequence $\bar{h}_{1:t}$ with probability $\bar{\mu}(\bar{h}_{1:t}|\mathcal{S}) = \prod_{a \in \mathcal{A}} \prod_{s \in \mathcal{S}} P_e^{\kappa|sa}$.

CTM Cost Function – Eq. (14.4.1)

$$\text{Cost}(\mathcal{S}|h_{1:t}) = \log \frac{1}{\bar{\mu}(\bar{h}_{1:t}|\mathcal{S})} + \Gamma_D(\mathcal{S}) = -\log \left(\prod_{a \in \mathcal{A}} \prod_{s \in \mathcal{S}} P_e^{\kappa|sa} \right) + \Gamma_D(\mathcal{S})$$

Plain English. $\Gamma_D(\mathcal{S}) := |\mathcal{S}| - 1 + |\{s : s \in \mathcal{S}, \ell(s) \neq D\}|$ is a **model penalty** for the state set $\mathcal{S}$ relative to the class $\mathcal{C}_D$ of context trees of maximal depth D ($\ell(s)$ is the depth/length of context s). Here $s_n = \phi_{\mathcal{S}}(h_{1:n})$, with $\phi_{\mathcal{S}}$ extracting the suffix of $h_{1:n}$ matching a state in $\mathcal{S}$. This exactly matches the earlier general cost function (14.3.3).

Context Tree Maximization: The Recursion

We rewrite *Cost* so that minimizing it reduces to finding the **CTM tree**. From (14.4.1), maximizing $2^{-\Gamma_D(S)} \bar{\mu}(h_{1:t}|S)$ is equivalent to minimizing *Cost*.

Context Tree Maximization – Eqs. (14.4.2)-(14.4.3)

$$P_{m,s}^D := \begin{cases} \frac{1}{2} \max \left\{ \prod_{a \in \mathcal{A}} P_e^{\kappa|sa}, \prod_i P_{m,\kappa_i s}^D \right\} & \text{if } \ell(s) < D \\ \prod_{a \in \mathcal{A}} P_e^{\kappa|sa} & \text{if } \ell(s) = D \end{cases}$$

$$S_{m,s}^D := \begin{cases} \bigcup_{\kappa_i} S_{m,\kappa_i s}^D \times \kappa_i & \text{if } \prod_a P_e^{\kappa|sa} < \prod_i P_{m,\kappa_i s}^D \text{ and } \ell(s) < D \\ \{\epsilon\} & \text{otherwise} \end{cases}$$

Plain English. At every context/node s we compare two options: **keep** s as a single state (probability $\prod_a P_e^{\kappa|sa}$), or **split** it into its children $\kappa_i s$ and recurse (probability $\prod_i P_{m,\kappa_i s}^D$). We take whichever is larger, times a $\frac{1}{2}$ “coding cost” for making the choice at this node at all (this is exactly the same recursive idea as CTM for binary sequence prediction in Section 5.5, generalized to a non-binary alphabet, with $[\prod_a P_e^{\kappa|sa}]^{-1}$ playing the role of the KT estimator there).

Lemma 14.4.4 and Theorem 14.4.5

Lemma 14.4.4 (Context Tree Maximization Equivalence)

For any state s , with $d = \ell(s)$, we have

$$P_{m,s}^D = 2^{-\Gamma_{D-d}(S_{m,s}^D)} \prod_{a \in \mathcal{A}} \prod_{u \in S_{m,s}^D} P_e^{\kappa|usa} = \max_{S \in \mathcal{C}_{D-d}} 2^{-\Gamma_{D-d}(S)} \prod_{a \in \mathcal{A}} \prod_{u \in S} P_e^{\kappa|usa}$$

(Proof left as Exercise 4; hint: see Section 5.5.)

Setting $s = \epsilon$ in this lemma gives the **CTM tree**, which minimizes the Cost function (14.4.1):

Theorem 14.4.5 (CTM Minimized Cost)

Given a history $h_{1:t}$, up to time t we have

$$\min_S \text{Cost}(S|h_{1:t}) = \log \frac{1}{P_{m,\epsilon}^D(e_{1:t}|a_{<t})} = \log \frac{1}{\bar{\mu}(h_{1:t}|S_{m,\epsilon}^D(h_{1:t}))} + \Gamma_D(S_{m,\epsilon}^D(h_{1:t}))$$

Plain English. $P_{m,\epsilon}^D$, computed at the root ϵ (empty context), automatically equals the best possible Cost over every candidate S – no explicit search needed.

Proof of Theorem 14.4.5 I

Proof.

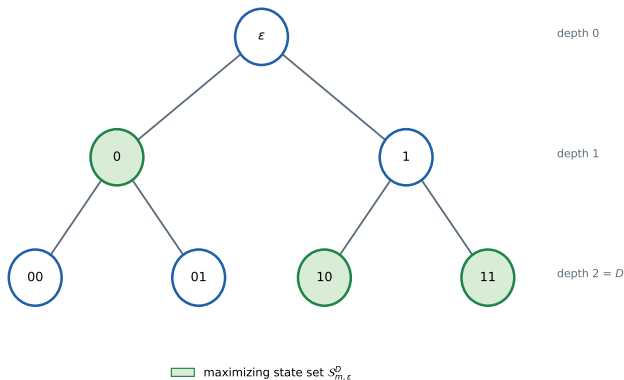
$$\begin{aligned}\min_S \text{Cost}(S|h_{1:t}) &= \min_S \log \frac{1}{\bar{\mu}(\bar{h}_{1:t}|S)} + \Gamma_D(S) \\&= \min_S \log \left(\frac{1}{\prod_{a \in \mathcal{A}} \prod_{s \in S} P_e^{\kappa|sa}} \right) + \Gamma_D(S) \\&= \min_S -\log \left(\prod_{a \in \mathcal{A}} \prod_{s \in S} P_e^{\kappa|sa} \right) + \Gamma_D(S) \\&= \max_S 2^{\log(\prod_{a \in \mathcal{A}} \prod_{s \in S} P_e^{\kappa|sa}) - \Gamma_D(S)} \\&= \max_S 2^{-\Gamma_D(S)} \left(\prod_{a \in \mathcal{A}} \prod_{s \in S} P_e^{\kappa|sa} \right) \\&= P_{m,\epsilon}^D(e_{1:t}|a_{<t}) \quad \blacksquare\end{aligned}$$

Proof of Theorem 14.4.5 II

The last line follows from Lemma 14.4.4 with $s = \epsilon$ (so $d = 0$). Since the recursions (14.4.2) can be computed efficiently (bottom-up, one pass over the tree), we now have an efficient method to find the optimal suffix state set $\mathcal{S}$ via (14.4.3), which minimizes the Cost function (14.4.1). This works well, provided the action and percept spaces are reasonably small for the KT-estimation to be good.

Picturing the Maximizing State Set

Context Tree Maximization: pruning to the maximizing state set



At each internal node, CTM asks: “is it worth splitting this context into its two children, or is this context already a good enough state on its own?” Propagating that decision from leaves to root yields the state set $\mathcal{S}_{m,\epsilon}^D$ that exactly minimizes the Cost function – the highlighted nodes above.

Python: A Toy Context Tree Maximization

We implement the recursion (14.4.2)-(14.4.3) directly on a depth-2 binary context tree with hand-picked “own” block probabilities $P_e^{\kappa|s_a}$ at every node (single action, so the product over $\mathcal{A}$ is just one number).

```
1 import math
2 D = 2
3 own_P = {"": .05, "0": .60, "1": .60, "00": .90, "01": .50, "10": .50, "11": .90}
4 memo = {}
5
6 def solve(node):                                # recursion (14.4.2)-(14.4.3)
7     if node in memo: return memo[node]
8     if len(node) == D:
9         P, S = own_P[node], [node]
10    else:
11        Pc0, Sc0 = solve(node+"0"); Pc1, Sc1 = solve(node+"1")
12        P, S = (0.5*Pc0*Pc1, Sc0+Sc1) if Pc0*Pc1 >= own_P[node] else (0.5*own_P[node], [node])
13    memo[node] = (P, S)
14    return P, S
15
16 P_root, S_root = solve("") # at s = eps (root)
17
18 # Output: "0": keep (P=.300); "1": keep (P=.300); root: SPLIT (P=.045)
19 # S = ['0', '1']; -log2(P)=4.4739 bits; Gamma_D(S)=1; Cost=5.4739 bits
```

Even though every depth-2 leaf is available, CTM decides the best trade-off is to stop splitting at depth 1 – landing on exactly the two-state space $\mathcal{S} = \{0, 1\}$ from Example 14.2.13.

Algorithm 14.3: CTMRL

Require: Environment μ ; learning loops m ; n_i 's, n_q ; minimal percept bits l_e

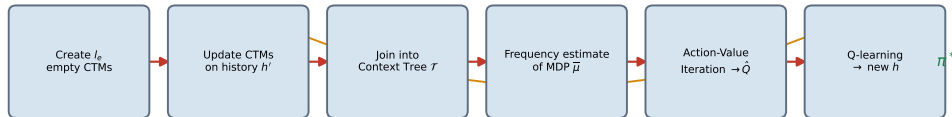
- 1: $i := 0$
- 2: Create l_e empty CTMs, where the i -th CTM predicts the i -th bit of the percept e
- 3: $h :=$ initial random history by performing n_0 random actions; $h' := h$
- 4: **while** $i < m$ **do**
- 5: Update the l_e CTMs based on history h'
- 6: Join learned contexts from each of the CTMs to form an Action-Observation Context Tree $\mathcal{T}$
- 7: Compute frequency estimates of the state transitions and reward probabilities of the MDP model based on states induced from tree $\mathcal{T}$ and history h
- 8: Use Action-Value Iteration to find an estimate $\hat{Q}$ of the optimal action values based on the frequency estimates
- 9: $Q := \hat{Q} + \frac{R_{max}}{1-\gamma}$
- 10: **if** $i < m - 1$ **then**
- 11: $h := \text{Q-learning}(Q, \mathcal{S}^S, \mathcal{A}, \mu, n_i)$; $h := [h, h']$; $i := i + 1$
- 12: **end if**
- 13: **end while**
- 14: $\hat{Q}' := \text{Q-learning}(Q, \mathcal{S}^T, \mathcal{A}, \mu, n_q)$
- 15: $\pi^*(s) := \arg \max_a \hat{Q}'(s, a)$ for all $s \in \mathcal{S}^T$
- 16: **return** π^*

What Algorithm 14.3 (CTMRL) Does

l_e separate CTMs handle the (binarized) percept bit-by-bit; they are joined into one Action-Observation Context Tree $\mathcal{T}$, which induces the MDP's states. R_{max} is the largest possible one-step reward; adding $\frac{R_{max}}{1-\gamma}$ to $\hat{Q}$ is an **optimism bonus** that encourages exploring under-visited states, echoing Chapter 9's optimistic agents.

Algorithm 14.3: the CTMRL pipeline

repeat while $i < m - 1$ (grow history, refine $\mathcal{T}$)



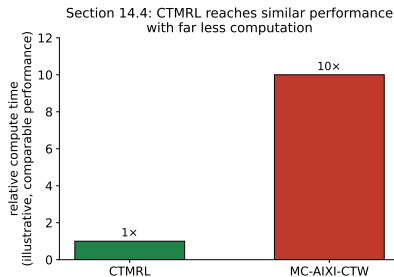
Binarization and CTMRL

For large action or percept spaces, the KT estimate will be poor. As in Chapter 12, we solve this by **binarizing** the action and percept spaces – the same trick as before.

Let l_e, l_a be the minimum number of bits needed to describe elements of the percept and action space respectively. CTMRL takes as input the environment, a number of learning loops, n_i (and n_q), and l_e . It starts with a CTM for each percept bit, then joins them into a single tree $\mathcal{T}$ containing $\mathcal{S}_{m,ae[1\dots i-1]}^{D+i-1}$ for all $i \leq l_e$ as subtrees. It estimates the state/reward transition probabilities of the MDP based on $\mathcal{T}$ via $\bar{\mu}$, updates the optimal action values with action-value iteration, then performs Q-learning using those values. After the learning loops complete, CTMRL performs Q-learning once more and defines $\pi^*(s) := \arg \max_a \hat{Q}^*(s, a)$.

Comparison to AIXI and Other Agents

CTMRL was compared to MC-AIXI-CTW (Section 12.3), as well as other agents like Φ MDP (Section 14.3), the classic U-Tree algorithm [McC96], and the more recent active LZ algorithm [FMRW10b], in a collection of environments detailed in Section 12.5.1. CTMRL achieved **comparable performance** with **drastically less computation time** required, with MC-AIXI-CTW taking on the order of **10 times** the amount of time CTMRL took to achieve similar near-optimal performance. This demonstrates that CTMRL is indeed a general, and that the FRL approach lends itself to more efficient approximations than AIXI approximations.



The AIXI vs. FRL/MDL Philosophy

The differences between CTMRL and MC-AIXI-CTW truly highlight the difference in approaches between the AIXI Bayesian approach and the MDL FRL approach:

- In the **AIXI approach**, we perform learning with $\xi_U \stackrel{\times}{=} M$ (a universal mixture that dominates the true measure M , up to a multiplicative constant) in theory, and **CTW** in practice; we do planning with **expectimax** in theory and **MCTS** in practice.
- In **FRL**, learning is done by finding/choosing the feature map ϕ in theory and the **maximal context tree** in practice (both theoretically and practically!), and expensive planning is replaced by **solving the MDP** via Bellman equations.

Whether explicit planning or value estimation leads to better policies is domain-dependent. Another potential advantage of FRL over AIXI is that Solomonoff's M models the *complete* history, which is expensive, while FRL only models those aspects that are separated out by the feature map.

Outline

- 1 Introduction: An Alternative to AIXI
- 2 14.1 Feature Reinforcement Learning Setup
- 3 14.2 History Aggregation beyond MDPs
 - 14.2.1 Surrogate MDP and Dispersion Probability
 - 14.2.2 (Q-)Value Inheritance for Fixed and Optimal Policy
 - 14.2.3 Extreme State Aggregation
- 4 14.3 Feature MDP
 - 14.3.1 Feature Learning
 - 14.3.2 Choices of the Cost Function
 - 14.3.3 Feature Dynamic Bayesian Networks
- 5 14.4 Context Tree Maximization Reinforcement Learning
- 6 14.5 Exercises
- 7 14.6 History and References

Exercises I

- ① [C25] Prove Theorem 14.2.12 (Value Inheritance).
- ② [C25] Prove Theorem 14.2.14 (Optimal Q-value Inheritance).
- ③ [C32] Using binarized sequential actions to improve Theorem 14.2.17 to $|\mathcal{S}| \leq O((\log |\mathcal{A}|)\varepsilon'^{-2}(1 - \gamma)^{-6})$. What is the cost of using binarized sequential actions? The better bound (14.2.18) is based on deeper insights into the structure of the problem [MH21a].
Hint: the reason for the remaining logarithmic dependence on $|\mathcal{A}|$ is that the discount γ must be replaced by $\gamma^{1/b}$ when $\mathcal{A}$ is sequentialized into b bits.
- ④ [C18] Prove Lemma 14.4.4 (Context Tree Maximization Equivalence).
- ⑤ [C35] Implement CTMRL and compare against MC-AIXI-CTW in several domains.

Plain English. The bracketed numbers [Cxx] are the book's standard difficulty ratings (roughly, “expect this to take about xx minutes if you have the right background,” scaled for research-level exercises).

Outline

- 1 Introduction: An Alternative to AIXI
- 2 14.1 Feature Reinforcement Learning Setup
- 3 14.2 History Aggregation beyond MDPs
 - 14.2.1 Surrogate MDP and Dispersion Probability
 - 14.2.2 (Q-)Value Inheritance for Fixed and Optimal Policy
 - 14.2.3 Extreme State Aggregation
- 4 14.3 Feature MDP
 - 14.3.1 Feature Learning
 - 14.3.2 Choices of the Cost Function
 - 14.3.3 Feature Dynamic Bayesian Networks
- 5 14.4 Context Tree Maximization Reinforcement Learning
- 6 14.5 Exercises
- 7 14.6 History and References

History and References I

Origins. The Feature Reinforcement Learning (FRL) approach, using state and state-action abstractions to solve the general reinforcement learning problem, started with Feature MDPs [Hut09e,Hut09f] and Feature Dynamic Bayesian Networks (DBNs) [Hut09d,Hut09a]. Since its inception there has been much work along this line, most of which has been included in this chapter. Comprehensive overviews can be found in [Ngu13,DSH14a,Das16,Maj21].

FRL theory. On the theoretical side, advances include using state abstractions for prediction and demonstrating their consistency [SH10]. Moving beyond the assumption that the abstracted process is an MDP was done in **Extreme State Aggregation** [Hut14a] (Section 14.2). Extreme State Aggregation was extended in [MH21b,MH21a], which binarized the action space to gain a double-exponential improvement on a bound on the required state space size. Action-state abstractions were investigated in [MH19], where the authors showed when the state-action abstraction can lead to near-optimal performance. The sequel to the original FRL papers [Hut09d,Hut09a] extended [Hut09e,Hut09f] to more complex structured/factored MDPs, specifically focusing on DBNs. [RLDG22] introduces a method able to give PAC guarantees in the episodic setting, assuming there exists a Markov abstraction.

History and References II

FRL practice. On the practical side, advances include using a practical implementation of Φ MDP to achieve performance competitive to MC-AIXI-CTW [NSH11], FRL with Context Tree Maximization [NSH12] (Section 14.4), FRL with looping suffix trees to solve the long-term dependency problem [DSH12], studying Q-learning in the general reinforcement learning setting [DSH13,MH18], and extending MC-AIXI-CTW by using state abstractions on the history [YZWN22]. Code for the approximations presented in this chapter is available from the book's website hutter1.net/ai/uaibook2.htm.

Regret results. There are many ways to measure the performance of an agent or abstraction; one is with **regret** (Section 8.1.4). In [MMR11] the authors introduced an algorithm for the FRL setting achieving regret $O(T^{2/3})$. This bound was improved when [NOR13] proposed an algorithm achieving regret $O(T^{1/2})$, which is optimal at least asymptotically. Both prior approaches relied on being given a finite set of abstractions, but [NMRO13] extended this to a countably infinite set. Many approaches consider *exact* abstractions; [OMR14] derived results for *approximate* abstractions, and [OPL+19] introduced a new algorithm achieving comparable regret bounds with possibly better bounds depending on the effective size of the induced state space.

History and References III

Solving MDPs with state abstractions. FRL focuses on solving the general RL problem through state abstractions; there is also much work on solving simpler MDP (and POMDP) problems this way. [SLO22] surveys many model-based methods for this problem. [LWL06] began the process of comparing five different unification approaches. Approximate abstractions are studied in [AHL16]. Continual/life-long learning faces an ever-growing history; state abstractions help alleviate this [AALL18]. Rate distortion theory was used in [AAA+19] to make the compression/value trade-off explicit. When the *action* space is very large, state-*action* abstractions are natural [MH19,AUK+20]. On the practical side, an exploration strategy based on feature representations gave state-of-the-art performance in Atari [MSEH17], and state abstractions reduced the size of sample-based search [HFD17].

History and References IV

Classical reinforcement learning. A general introduction to classical state-based RL can be found in [SB18] (see Section 6.9 for more advanced books). It covers multi-armed bandits (Section 12.2.3), finite MDPs (Chapter 11), Bellman equations (Section 14.1), MCTS planning (Section 12.2.4), $TD(\lambda)$ [Sut88,HL07], $Q(\lambda)$ [WD92,PW94,DSH13,MH18], SARSA [RN94], approximation methods, policy gradients [Wil92,KHS01a], applications, and dichotomies: off- vs. on-policy, exploration vs. exploitation, model-based vs. model-free.

Other RL ideas. The most widely used abstractions in RL are not discrete state/history aggregations, but approximations based on continuously parameterized (Q-)Value functions. Even for *linear* function approximation, convergence is hard to come by: model-free TD-style algorithms cannot be guaranteed to converge to the correct solution unless the linear basis functions are essentially state aggregators [HYZM19]. Model-based methods and brilliant “unnatural” algorithms like ETD [SMW16] have convergence guarantees for linear function approximation. For state aggregation, [MH18] shows Q-learning converges even if the aggregated dynamics is non-Markov, as in Section 14.2. [HL07] derives a unique state-dependent learning rate for $TD(\lambda)$ via a variational principle and bootstrapping.

History and References V

Rather than hard aggregation or smooth function approximation, one can also impose or use a **metric** on the observation or state space, natural in RL for robotics [Thr02]. [ZGHS06] extends [McC96] to general metrics over histories and hence continuous POMDPs, with an RL algorithm that learns directly on a mobile robot.

An unorthodox RL approach is to learn the sequence of actions most likely to cause the agent to transition from state s to s' – these **inverse MDP models** automatically only model the action-relevant aspects of the world [HH22]. With this idea in mind, [OHC+20] presented a method to meta-learn the update rule used, instead of a fixed update rule. **Imagination-based planning** was introduced in [PLV+17] to let agents construct and evaluate plans without taking real actions; an alternative imagination approach was studied in [SvHH+16], which learned a model for planning purposes only (with possibly different state, action, reward space, and time steps).

History and References VI

In [OIO19] a simple way to interpret RL as inference was established. **Distributional RL** [BDM17,BDR23,WDA+23] uses Bellman equations [Bel57] for the *distribution* of returns, and argues why this improves performance even when we only care about expected returns. Related is **Upside-down RL** [Sch19a]. RLAdvice [DSH14b] is an imitation learning algorithm based on (Bayesian) DAgger, using value estimates from UCT as advice, applied to three Atari games from the Arcade Learning Environment (ALE).

Other AGI approaches. There are many approaches to building artificial general intelligences [GP07,AGI08], with [SBP22,LeC22] two prominent recent examples.